
EMPLOYMENT MANAGEMENT SYSTEM

Authentication

JWT, the security filter chain, and the four auth flows
LOGIN, REGISTER, SESSION, LOGOUT • 2 OCTOBER 2026

Contents

01	1. The mental model	3
02	2. The pieces	3
03	3. <code>User</code> is both a database row and a Spring Security object	4
04	4. <code>JwtService</code> - creating and reading tokens	5
05	5. <code>JwtAuthenticationFilter</code> - the most important class to explain	6
06	6. The two repository queries	7
07	7. Flow A - Log in	8
08	8. Flow B - Register	9
09	9. Flow C - Restore the session on reload ("who am I?")	11
10	10. Flow D - Log out	12
11	11. Frontend storage details worth explaining	13
12	12. Full auth endpoint summary	14

How the app knows who you are, and how it proves it on every later request.

1. The mental model

There is **no server-side session**. When you log in, the backend signs a JSON Web Token (JWT) and hands it back. The frontend stores it in `localStorage`. Every later request sends it back in the `Authorization` header, and a servlet filter re-establishes your identity from it.

```
POST /api/auth/login { username, password }
    |
    v
verify password with BCrypt
    |
    v
sign JWT → { "token": "eyJhbGciOi ... " } → localStorage["ems_token"]

then, every request:
GET /api/employees/paged
Authorization: Bearer eyJhbGciOi ...
    |
    v
JwtAuthenticationFilter verifies signature + expiry, loads the User from the DB,
puts it in SecurityContext → @PreAuthorize can now read your roles
```

2. The pieces

File	Responsibility
<code>security/JwtService.java</code>	create tokens, read claims, validate expiry
<code>security/JwtAuthenticationFilter.java</code>	runs on every request; turns the header into an authenticated <code>SecurityContext</code>
<code>security/CustomUserDetailsService.java</code>	<code>username → User</code> from the database
<code>security/config/SecurityConfig.java</code>	which URLs are public, how errors are returned, where the filter goes
<code>user/User.java</code>	the login account, and the <code>UserDetails</code> object Spring Security uses
<code>user/UserRepository.java</code>	the two fetch queries used during auth
<code>auth/AuthController.java</code>	<code>POST /login</code> , <code>POST /register</code> , <code>GET /me</code>
<code>auth/AuthServiceImp.java</code>	login logic + "who am I"

File	Responsibility
<code>auth/UserRegistrationFacade.java</code>	registration logic (it is <i>not</i> in <code>AuthService</code>)
<code>auth/dto/*</code>	<code>LogInRequest</code> , <code>RegisterRequest</code> , <code>AuthResponse</code> , <code>CurrentUserResponse</code>

3. User is both a database row and a Spring Security object

`User.java:23` declares:

```

@Entity
@Table(name = "users")
public class User implements UserDetails {

```

JAVA

`UserDetails` is a Spring Security interface with five methods. `User` fills them in:

- `getAuthorities()` (line 60) - **this is the important one**. It converts each `Role` into a `SimpleGrantedAuthority` whose name is the role string, e.g. `"ROLE_ADMIN"`. This collection is what `@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")` and `hasRole('EMPLOYEE')` are evaluated against.
- `getPassword()` - Lombok generates it from the `password` field; it returns the BCrypt **hash**, which is what Spring compares against.
- `getUsername()` - Lombok generates it.
- `isAccountNonExpired()`, `isAccountNonLocked()`, `isCredentialsNonExpired()` - all hard-coded `true` (lines 65, 70, 75). The app never expires or locks an account.
- `isEnabled()` - Lombok generates it from the `enabled` column, and `UserServiceImp` sets it to `true` on creation.

There is also a convenience method that is *not* part of `UserDetails`:

```

public String getEmail(){ return employee != null ? employee.getEmail() : null; }

```

JAVA

A `User` has no `email` column - the email lives on `Employee`. This getter is how `GET /api/auth/me` can return an email without a second lookup.


```

    return Jwts.parserBuilder()
        .setSigningKey(getSignInKey())
        .build()
        .parseClaimsJws(jwt)
        .getBody();
}

```

`parseClaimsJws` **verifies the signature and the expiry**. If the token was signed with another key, was tampered with, or is expired, it throws. That is why `JwtAuthenticationFilter` wraps this call in `try/catch` - an unparseable token should result in "anonymous", which later becomes a `401`, not a `500`.

Validating (`isTokenValid`, line 65)

```

final String username = extractUsername(jwt);
return (username.equals(userDetails.getUsername()) && !isTokenExpired(jwt));

```

JAVA

Two checks: the username inside the token must match the account loaded from the database, and the token must not be expired.

5. `JwtAuthenticationFilter` - the most important class to explain

`OncePerRequestFilter` guarantees it runs once per request. It is registered *before* `UsernamePasswordAuthenticationFilter` in `SecurityConfig:53`.

```

final String authHeader = request.getHeader("Authorization");

if (authHeader == null || !authHeader.startsWith("Bearer ")) {
    filterChain.doFilter(request, response); // no token: continue as anonymous
    return;
}

String token = authHeader.substring(7); // strip "Bearer "
String username;
try {
    username = jwtService.extractUsername(token);
} catch (Exception e) {
    filterChain.doFilter(request, response); // bad/expired token: continue as anonymous
    return;
}

if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
    UserDetails userDetails = customUserDetailsService.loadUserByUsername(username);

    if (jwtService.isTokenValid(token, userDetails)) {
        UsernamePasswordAuthenticationToken authenticationToken =
            new UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities());
    }
}

```

JAVA

```

        authenticationToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

        SecurityContextHolder.getContext().setAuthentication(authenticationToken);
    }
}
filterChain.doFilter(request, response);

```

Four situations:

1. **No `Authorization` header** - the filter is a no-op. The request continues anonymous and is either permitted (`/api/auth/**`) or rejected with `401`.
2. **`Bearer` prefix missing** - same as above.
3. **Header present but the token is invalid/expired** - the `catch` swallows the exception and the request continues anonymous. This means a bad token never produces a `500`.
4. **Valid token** - the user is loaded **fresh from the database** (`loadUserByUsername`), re-validated, and stored in `SecurityContextHolder`.

Point 4 is the part people get wrong. The token carries a `roles` claim, but the filter **ignores it** and uses `userDetails.getAuthorities()`, which is read from the `roles` join table. That is deliberate: changing a user's role in the database takes effect on their very next request, with no need to wait for the old token to expire. (The `roles` claim in the token is therefore effectively decorative.)

Two more details:

- `new UsernamePasswordAuthenticationToken(userDetails, null, authorities)` - the 3-argument constructor means "already authenticated" (`authenticated == true`). The 2-argument version would mean "credentials not yet verified".
- `credentials` is `null` because there is nothing to check after the token was verified.

6. The two repository queries

```

@Query("Select u From User u Left Join Fetch u.roles Where u.username =:username")
Optional<User> findByUsernameWithRoles(String username);

@Query("Select u From User u Left Join Fetch u.roles Left Join Fetch u.employee Where u.username =:username")
Optional<User> findByUsernameWithDetails(String username);

```

- `Left Join Fetch` performs the join **and** loads the related rows in the same query. This is needed because `roles` is mapped `FetchType.EAGER`, but an eager `Set` on a `@ManyToMany` combined with pagination would otherwise be a problem, and because `employee` is `LAZY` - the second query is the only way to read it safely.

- `findByUsernameWithRoles` -> used by the JWT filter and therefore on **every** request. It does not fetch the employee, which keeps it cheap.
- `findByUsernameWithDetails` -> used only by `GET /api/auth/me`.

7. Flow A - Log in

Screen: `src/pages/LoginPage.jsx`

```

user types username + password, clicks "Sign in"
handleSubmit (line 21)
  e.preventDefault()           stop the browser's own form submit
  setSubmitting(true)          disable the button, show "Signing in ..."
  loginUser({ username, password })  api/http.js:112
    POST /api/auth/login  body: JSON
  on success:
    setSession(token)          authStore: writes localStorage + state
    await loadUser()           GET /api/auth/me, fills user state
    navigate("/")              go to the Dashboard
  on failure:
    pushToast(error.message, "error")  red toast from uiStore
  finally:
    setSubmitting(false)

```

`if (token) return <Navigate to="/" replace />;` (line 19) means an already-logged-in visitor who opens `/login` is bounced to the app.

Backend:

1. `AuthController.login` (line 27). `@Valid @RequestBody LoginRequest` means the body must be valid JSON and deserialise into `LoginRequest`. `LoginRequest` has **no** validation annotations, so an empty body fails at JSON parsing (`400 Malformed request body`) but a blank username is caught later by authentication.
2. `AuthServiceImp.login` (line 33):

```

Authentication auth = authenticationManager.authenticate(
    new UsernamePasswordAuthenticationToken(username, password));

```

`authenticationManager` is the bean exposed in `SecurityConfig:58-61`

(`AuthenticationConfiguration.getAuthenticationManager()`). It runs Spring's standard chain, which ends in a `DaoAuthenticationProvider`:

- `CustomUserDetailsService.loadUserByUsername(username)` -> `401`-ish `UsernameNotFoundException` if the user does not exist.
- `BCryptPasswordEncoder.matches(rawPassword, storedHash)`.

- checks `isEnabled`, `isAccountNonLocked`, etc.

If the password is wrong, `BadCredentialsException` is thrown and `GlobalExceptionHandler` converts it to **401 "Invalid username or password"**. Note the message is deliberately vague - it does not tell you whether the username exists.

3. Back in `AuthServiceImp`:

```

UserDetails userDetails = (UserDetails) auth.getPrincipal();
String token = generateToken(userDetails);
return new AuthResponse(token);

```

JAVA

`auth.getPrincipal()` is the `User` entity loaded in step 2, so `getAuthorities()` gives the roles that go into the token.

4. Response `200 {"token":" ... "}`. Login is **not** an endpoint that sets a cookie - the token is the only thing returned.

8. Flow B - Register

Screen: `src/pages/RegisterPage.jsx`. The hint text is important: *"Email must match an existing employee"* - that is the actual rule.

Backend: `AuthController.register` -> `201 Created` + token.

`UserRegistrationFacade.register` (line 24) runs four steps, each for a reason:

```

// 1- Check if user already exists
if (userService.existsByUsername(request.getUsername())) {
    throw new DuplicateResourceException("This username already exists");
}

// 2- load Employee
Employee employee = employeeService.getEmployeeByEmail(request.getEmail());

// 3- load Role
Role role = roleService.getEmployeeRole();

// 4- create user
User user = userService.createUser(request.getUsername(),
                                   request.getPassword(), role, employee);

return new AuthResponse(authService.generateToken(user));

```

JAVA

1. **Uniqueness is case-insensitive.** `UserRepository.existsByUsernameIgnoreCase` is used, so `John` and `john` are the same account. Failing this throws `DuplicateResourceException` -> **409 Conflict**.

2. The email is not stored on `User`. It is looked up on `Employee` and a missing match throws `ResourceNotFoundException` -> **404**. So you cannot register an account for an email that has no employee record. That is the mechanism that stops anybody from self-registering as somebody else: the email must already belong to a real employee record, and the new account is bound to that employee.
3. The role is always `ROLE_EMPLOYEE`. `RoleServiceImpl.getEmployeeRole()` looks up `ROLE_EMPLOYEE` by name; if the `roles` table was never seeded it throws `ResourceNotFoundException`. There is no way to register as admin or manager - this is the single most important security fact about registration.
4. `UserServiceImp.createUser` (line 20):

```

if (existsByUsername(username)) throw new DuplicateResourceException("Username already exists");
User user = new User();
user.setUsername(username);
user.setPassword(passwordEncoder.encode(rawPassword)); // BCrypt hash, never the raw password
user.setEnabled(true);
user.getRoles().add(role);
user.setEmployee(employee);
return userRepository.save(user);

```

The duplicate check runs **again** here. That is intentional defence in depth: the facade check gives a friendlier message, this one protects the service if it is ever called from somewhere else.

DTO validation (`RegisterRequest`):

Field	Rule
username	@NotBlank
password	@NotBlank, @Size(min = 6, message = "Password size must be minimum of 6")
email	@NotBlank, @Email(message = "invalid email format")

`@Valid` on the controller parameter makes Spring check these *before* the method body runs. A violation throws `MethodArgumentNotValidException`, which `GlobalExceptionHandler` turns into **400** with a `fieldErrors` map like:

```

{ "status": 400, "message": "Validation failed: 1 error(s)",
  "errors": { "password": "Password size must be minimum of 6" } }

```

The frontend's `api()` throws `new Error(errorData.message)`, so the toast shows `"Validation failed: 1 error(s)"`. (The per-field detail is available in the response but the UI does not display it - a small known gap.)

`UserService` is also where `enabled` is set; the column would otherwise default to whatever MySQL decides, which is why `createUser` sets it explicitly.

9. Flow C - Restore the session on reload ("who am I?")

This is the flow people forget, and it is why `GET /api/auth/me` exists.

Problem: after `f5`, the token is still in `localStorage`, but `user` (the object with the roles) is gone from memory. Without roles the UI would have to log you out on every refresh.

`RequireAuth` (`src/components/RequireAuth.jsx`):

```
const token = useAuthStore((s) => s.token);
const user = useAuthStore((s) => s.user);
const loadUser = useAuthStore((s) => s.loadUser);

useEffect(() => {
  if (token && !user) loadUser();
}, [token, user, loadUser]);

if (!token) return <Navigate to="/login" replace />;
if (!user) return <p className="p-8 text-slate-500">Loading ... </p>;
return <Outlet />;
```

Three states, in this exact order:

State	Meaning	What renders
<code>!token</code>	never logged in	redirect to <code>/login</code>
<code>token && !user</code>	token present, identity not yet confirmed	<code>Loading ...</code>
<code>token && user</code>	ready	<code><Outlet /></code> - the nested routes

The middle state is essential. If the second line were missing, a page refresh would flash the redirect to `/login` before `loadUser` finished - this was an actual bug that was fixed.

`loadUser` (`src/store/authStore.js:15`):

```
loadUser: async () => {
  const token = get().token;
  if (!token) return;
  try {
    const response = await fetch("/api/auth/me", {
      headers: { Authorization: `Bearer ${token}` },
    });
    if (response.ok) set({ user: await response.json() });
    else if (response.status === 401) get().logout();
  } catch {}
}
```

- It calls `fetch` directly rather than the `api()` wrapper on purpose: this runs during boot, and it must not trigger `api()`'s global 401 redirect logic.

- `401` -> `logout()` clears the token, which makes `RequireAuth` redirect on the next render.
- Any other non-ok status is ignored, so the app stays in `Loading ...`.
- `catch {}` swallows network errors - a deliberate UX choice (offline -> stuck on "Loading..." rather than a crash).

Backend: `AuthController.me` (line 40) takes the `Authentication` that the JWT filter put in the `SecurityContext` and calls `authService.getCurrentUser(authentication.getName())`.

```

User user = userRepository.findByUsernameWithDetails(username) ... ;
List<String> roles = user.getRoles().stream().map(Role::getName).sorted().toList();
Long employeeId = user.getEmployee() != null ? user.getEmployee().getId() : null;
return new CurrentUserResponse(user.getUsername(), user.getEmail(), roles, employeeId);

```

JAVA

Response:

```

{ "username": "jane", "email": "jane.smith@example.com",
  "roles": ["ROLE_EMPLOYEE"], "employeeId": 1 }

```

JSON

Four things worth noting:

- `.sorted()` gives a stable role order, so the header always shows the same role.
- `getEmail()` is the `User` helper from section 3, which reads through to `employee.email`. It returns `null` for a manager account with no employee row.
- **`employeeId` is the linchpin of employee attendance.** The frontend never sends its own id when clocking in; it uses this value, and `AttendancePage` derives `effectiveClockEmployeeId = String(user?.employeeId ?? "")` from it. The server ignores the client's value anyway (section 4 of `05-attendance.md`).
- The `authentication == null` branch returning `401` is unreachable in practice: `SecurityConfig` already rejects unauthenticated requests before the controller.

10. Flow D - Log out

```

click "Logout" (Layout.jsx:29)
  handleLogout()
    queryClient.clear()      drop every cached TanStack Query result
    logout()                 localStorage.removeItem("ems_token"); set({token:null,user:null})
    navigate("/login")

```

TEXT

Why `queryClient.clear()` first: TanStack Query's cache is in memory only, so it would normally vanish on reload - but without clearing it, the *next* user who logs in on the same browser tab could briefly see the previous user's cached data before refetching. Clearing it removes that window.

Why logout needs no backend call: JWTs are self-contained and stateless. There is nothing to invalidate server-side. The token stays valid until it expires, so a stolen token would still work - see the notes in `07-security-model.md` about token lifetime and logout not revoking tokens.

The header (`Layout.jsx:73-92`) renders `user.username` and strips the prefix for display: `user.roles[0].replace("ROLE_", "")` turns `ROLE_MANAGER` into `MANAGER`. That is also why `getCurrentUser` sorts the roles - with more than one role the first element would otherwise be arbitrary.

11. Frontend storage details worth explaining

`src/store/authStore.js`:

```
token: localStorage.getItem("ems_token") || null,
user: null,
```

JS

- The token is **read from** `localStorage` **when the store is first created**, which is what makes a page refresh keep you logged in.
- `user` deliberately starts as `null` even if a token exists, so `RequireAuth` always re-validates against the server instead of trusting stale roles.
- `setSession(token)` writes to `localStorage` **and** to Zustand state; React re-renders immediately from state, and the token survives a refresh.
- Zustand selectors are used everywhere (`useAuthStore((s) => s.token)`) so a component only re-renders when the field it uses changes.

`src/api/http.js:4` - the wrapper every API call goes through:

```
const token = useAuthStore.getState().token;
const headers = {
  ... (options.body ? { "Content-Type": "application/json" } : {}),
  ... (token ? { Authorization: `Bearer ${token}` } : {}),
};
const response = await fetch(path, { ...options, headers });

if (token && response.status === 401) {
  useAuthStore.getState().logout();
  if (!window.location.pathname.startsWith("/login")) window.location.href = "/login";
}
if (!response.ok) {
  const errorData = await response.json().catch(() => null);
  throw new Error(errorData?.message || `Request failed (${response.status})`);
}
if (response.status === 204) return null;
return response.json();
```

JS

Details:

- `useAuthStore.getState()` is the **non-hook** way to read the store - required because this is not a React component.
- `Content-Type` is only set when there is a body. That is exactly why the check-in/check-out calls (`method: "POST"` with no body) work, and why a `curl` that forces `application/x-www-form-urlencoded` gets a `500` from Spring's `HttpMediaTypeNotSupportedException` - which `GlobalExceptionHandler` does not handle specially, so it falls into the generic `Exception` handler.
- `401` + a token present = the token was rejected: log out and hard-redirect with `window.location.href` (a full page load, as opposed to the router's `navigate`). This is the global "your session died" handler.
- `204` -> `null`, so `deleteEmployee` does not try to parse an empty body.
- `catch(() => null)` on `response.json()` covers error responses that are not JSON (for example the plain-text 500 above).

12. Full auth endpoint summary

Method	Path	Auth required	Body / Params	Returns	Errors
POST	<code>/api/auth/login</code>	No	<code>{username, password}</code>	<code>200 {token}</code>	400 malformed body, 401 bad credentials
POST	<code>/api/auth/register</code>	No	<code>{username, password, email}</code>	<code>201 {token}</code>	400 validation, 404 email has no employee, 409 username taken, 404 <code>ROLE_EMPLOYEE</code> missing
GET	<code>/api/auth/me</code>	Yes	-	<code>200 {username, email, roles[], employeeId}</code>	401 invalid/missing token

All three are matched by `.requestMatchers("/api/auth/**", "/error").permitAll()` in `SecurityConfig:35`, which means `/me` is *also* publicly permitted at the URL level. It still returns `401` for a missing token because the controller requires an `Authentication` argument, and it returns `401` for a bad token because the filter leaves the context anonymous. Being in `permitAll` means the filter chain does not reject the request early - it does not make `/me` public.